# AXIPAYS

## Technical Assignment

| Type | Issued by |
| --- | --- |
| **Technical Task** | **Team AXIPAYS** |

## Section 1 — Payment Checkout Page

Build a clean, professional payment checkout page that collects all necessary card and billing details. The UI should feel trustworthy, responsive, and user-friendly.

### Required Fields

- Card Holder Name
- Email Address
- Card Number (masked display)
- Card Expiry - Month & Year
- Card CVV / CVC (masked input)
- Payment Amount
- Currency
- Country
- Address
- Phone

## Section 2 — API Integration

Integrate the payment initiation API and handle the response lifecycle correctly, including redirection flow.

## Initiate Payment Endpoint

| | |
|---|---|
| **Endpoint URL** | https://payment-assignment.onrender.com/initiate-payment |

| | |
|---|---|
| **Method** | POST |

### Header

| | |
|---|---|
| **Content-Type** | application/json |
| **Hash** | Your encrypted hash |

## Security - HMAC-SHA256 Hash Header

The request must include a Hash header computed using HMAC-SHA256 (uppercase hex output).

Steps to generate hash:

- Extract first 6 digits and last 4 digits of the card number
- Concatenate: first6 + last4 (10-digit string)
- Reverse the combined 10-digit string
- Reverse the email address string
- Build the message: reverse(email) + "AXIPAYS" + reverse(first6+last4)
- Convert it to uppercase
- Encode uppercase result with secret key AXI2026 & use as the Hash header value

## Response Handling

On a successful API response, you will receive a redirection_url field. Your implementation must:

- Parse the response JSON
- Extract the redirection_url
- Open or redirect the browser to that URL
- You will get a final response of Success, Failed, Pending
- Show a modal window according to final status

## Bonus:- Handle redirection_url in both browser window and iframe

## Section 3 — Dashboard Page

Build a visually appealing dashboard that surfaces key transaction metrics and a detailed transaction history table. Use charts and visual components to make data easy to understand.

### Dashboard Features

- Get all transaction details by below get transactions API.
- Summary cards -Total Transactions, Total Success Volume, Total Success Count, Total Failed count which includes Failed + Pending.
- Charts - Transaction status breakdown, volume over time , currency distribution (Donut Chart)
- Responsive layout - adapts cleanly on desktop and mobile

### Transaction History Table

Fetch all transactions from the backend and display them with pagination.

| | |
|---|---|
| **API URL** | https://payment-assignment.onrender.com/transactions?page=1&limit=100 |

| | |
|---|---|
| **Method** | GET |

### Required Table Columns

| Column | Notes |
|---|---|
| **Order ID** | Unique transaction identifier |
| **Card Number** | Masked- show first 6 and last 4 only |
| **Email** | Cardholder email |
| **Expiry Month & Year** | e.g., 08 / 2027 |
| **Card CVC** | Always masked - show *** only |
| **Amount** | Transaction value |
| **Currency** | e.g., USD, EUR, GBP |
| **Status** | Pending / Success / Failed with colour badge |

## Key Things We'll Look For

| | |
|---|---|
| **Luhn Algorithm** | Validate the card number using the Luhn check before submission. Look it up if you're not familiar this is standard in the industry. |
| **Card Data Masking** | Always mask sensitive fields  card number and CVV on both the checkout and transaction history pages. No raw values should ever be visible. |
| **API Handling** | Cleanly handle payment processing and the redirect/callback flow. |
| **Security First** | No sensitive data leaks. Encryption and secure handling are must-haves, even in a task project. No corners cut. |

## Submission

**Format:**

- Submit your work via a public GitHub repository, zip file and the live deployed link. Share the links in your reply to this email.
- Include a **README.md** file explaining the code structure, flow, and any key decisions or assumptions made.

## Tech Stack

- Use React framework or JS
- UI should be clear and functional with responsiveness.
- It will be a bonus to share the live deployed link of the project.

## Bonus - Extra Points

- Smooth UI animations - loading indicators, status transition effects, animated button states
- Fully responsive layout - looks great on all screen sizes
- Polished, professional feel - pixel-perfect and consistent design system
- Creative enhancements - go beyond the spec if you have a better idea for UX/UI or security

## Important Notes

**Think like a real developer, not a robot.**

Structure your code neatly, follow best practices, and use good naming conventions.

**Focus on quality over quantity.**

A clean, fully working basic version beats a messy advanced one every time.

Be creative — you can go beyond what's asked if you have a better idea to enhance UX/UI or security!

## Contact & Queries

For any technical questions regarding this assignment, feel free to reach out.

**Aditya Bodani WhatsApp (Text Only): +91 8602133348**

*Please do not call - text messages only.*

*Good luck! We're excited to see how you approach real-world problems like a true engineer.*

**Bring your A-game!**

**Team AXIPAYS**